

Inside foc_isr()

Every calculation in the 16 kHz control ISR: the unit system, the fixed-point representations, the algebra behind each transform, an end-to-end worked example with real numbers, and the overflow and precision analysis that says why each expression is written the way it is.

1. The unit system, and why it is what it is

Choosing units badly is how a control loop acquires scaling constants nobody can justify. Three rules fix them here.

- **Physical quantities are in integer engineering units** - milliamps, millivolts, rpm - not in normalised per-unit. A number in the debugger is then directly comparable with a meter reading, and a wrong sign or a factor of ten is visible instead of plausible.
- **Angles are a 32-bit accumulator** where the full range is one electrical revolution. Wraparound becomes integer overflow: free, no branch, no comparison, and no rounding drift accumulating over hours.
- **Gains and trigonometric values are Q15**, so a multiply-and-shift replaces a division and the intermediate stays inside 32 bits when one operand is a physical quantity.

Quantity	Type	Unit / scale	Resolution	Range
Phase current	int32_t	milliamps	2.52 mA (one ADC count)	+/- 82.5 A at full scale
Bus and phase voltage	int32_t	millivolts	1 mV nominal	set by the divider
Electrical angle	angle_t (uint32_t)	2^32 = 360 deg electrical	0.0000000838 deg	one revolution, wraps
Mechanical position	uint32_t	2^32 = one revolution	2.47 arcsec at 19-bit	one revolution, wraps
Speed	int32_t	rpm	0.46 rpm over 4 periods	+/- 2 billion, unused
Time	uint32_t	ticks of 240 MHz	4.167 ns	wraps every 17.9 s
Duty	uint16_t	counts of ARR = 7500	4.167 ns of on-time	0 to CCR_MAX = 7167
Gains, sin/cos	int32_t / q15_t	Q15, 32768 = 1.0	1/32768	+/- 1.0

Why milliamps and not amps or per-unit. An ADC count is 2.52 mA, so milliamps carry the full converter resolution in an integer with no scaling factor to remember. Amps would quantise to nothing; per-unit would need the full scale current to appear in every expression, and it is exactly the kind of constant that gets changed in one place and not another.

Why the angle is not degrees. In degrees, every use needs a modulo and the advance term needs a conditional wrap. As a 32-bit accumulator, adding the advance IS the wrap, converting mechanical to electrical is a multiply that overflows exactly the right number of times, and the trigonometric table index is a shift. Three operations disappear rather than being optimised.

2. Acquisition: ADC counts to milliamps

The signal chain is a low-side shunt, an INA241 amplifier referenced to mid-scale, and a 16-bit converter. Working backwards from counts:

```
V_shunt = I * R_shunt           Ohm's law across the shunt
V_out   = V_ref/2 + V_shunt * GAIN  INA241, bidirectional
counts  = V_out * 65536 / V_ref     16-bit conversion
```

so, with $\text{delta} = \text{counts} - \text{offset}$:

```
V_shunt_uV = delta * VREF_MV * 1000 / (65536 * GAIN)
I_mA       = V_shunt_uV / R_mOHM
```

The last step uses an identity worth stating explicitly, because it is what keeps the whole conversion in integers: **$I(\text{mA}) \times R(\text{milliohm}) = V(\text{microvolt})$** , exactly. No scaling factor and no rounding.

Folding the two lines together:

```
I_mA = delta * VREF_MV * 1000 / (65536 * GAIN * R_mOHM)

MA_PER_COUNT_Q16 = VREF_MV * 1000 / (GAIN * R_mOHM)
I_mA             = (delta * MA_PER_COUNT_Q16) >> 16
```

The 2^{16} of the Q16 scaling and the 65536 of the ADC full scale cancel. That is not a coincidence to exploit quietly - it is the reason the constant fits in 32 bits at all.

Writing it unreduced overflows at compile time. $VREF_MV * 1000 * 65536$ is $2.16e11$, which does not fit in `int32_t`, and the compiler silently produces a wrong constant. GCC warns with `-Woverflow`; without that flag this is a drive whose currents read 7x low with no other symptom.

With the placeholder constants $VREF = 3300 \text{ mV}$, $GAIN = 20$, $R = 1 \text{ milliohm}$:

Expression	Value	Meaning
<code>MA_PER_COUNT_Q16</code>	165000	Q16 multiplier
$165000 / 65536$	2.5177	milliamps per ADC count
$32768 * 2.5177$	82500 mA	full-scale current, +/- 82.5 A
$82500 / 18000$	4.58	headroom over the 18 A rating

Two things the ADC path does that are easy to miss

The offset is measured, not assumed. A bidirectional amplifier reads near half scale at zero current, but not exactly half - the reference, the amplifier offset and the converter offset all contribute and all drift. `hal_init_adc()` averages 256 conversions with the bridge off. It also has to be measured before the watchdog thresholds are armed, because those are absolute counts relative to this value.

The sign is inverted. A low-side shunt sees current leaving the phase into the negative rail, so the sensed polarity is opposite to motor convention. `CURRENT_SIGN` is a named constant because stage 3 has *sign of every sensed current confirmed* as a pass criterion, and flipping it must be a one-character edit.

A precision defect in the bus voltage path

```
out->v_bus_mv = (raw * ADC_VREF_MV / 65536) * VBUS_DIV_NUM / VBUS_DIV_DEN;
```

The division happens before the multiplication, so the intermediate is truncated to whole millivolts at the ADC input and then multiplied by the divider ratio. With a 20:1 divider the result is quantised to 20 mV rather than 1

mV - twenty times worse than the converter.

Multiplying first would be correct but overflows: $47663 \times 3300 \times 20$ is $3.15e9$ against an int32 ceiling of $2.15e9$.
The fix is a 64-bit intermediate:

```
out->v_bus_mv = (int32_t)(((int64_t)raw * ADC_VREF_MV * VBUS_DIV_NUM)
                          / (65536 * VBUS_DIV_DEN));
```

3. Encoder: process data to electrical angle

The iC-MBE returns the position right-aligned in its process data - 19 bits for a 32/31 Nonius target. Two operations turn that into the angle the transforms use.

3.1 Left-align into the position accumulator

```
position = pd << (32 - ENC_RESOLUTION_BITS);    /* 19-bit: << 13 */
```

After this, **position spans one MECHANICAL revolution over the full uint32 range**, whatever the encoder's native resolution. A 19-bit and a 20-bit encoder become indistinguishable upward, and the difference of two positions is wrap-safe by the same argument as the angle.

The low 13 bits are always zero, so the effective resolution is unchanged - $2^{19} = 524288$ counts per revolution, or 2.47 arcsec. What the shift buys is that L2 never has to know the resolution.

3.2 Mechanical to electrical

```
angle = position * MOTOR_POLE_PAIRS + COMMUTATION_OFFSET;
```

This is the operation the whole representation exists for. With 4 pole pairs, multiplying a value that spans one mechanical revolution by 4 produces a value that overflows four times per revolution - which is exactly four electrical revolutions. **The overflow IS the modulo.** No conditional, no comparison, no accumulated rounding.

Two different pole-pair counts live in this project and they are unrelated. MOTOR_POLE_PAIRS is a property of the motor's magnet arrangement and sets the electrical-to-mechanical ratio. MPC is the pole-pair count of the encoder's magnetic ring and sets the encoder's resolution and its maximum speed. Confusing them gives a drive that commutates at the wrong rate and looks like a wiring fault.

4. Speed estimate

```
d = (int32_t)(position - prev_position);    /* wrap-safe */

revolutions = d / 2^32
seconds      = interval_ticks / HAL_TICK_HZ
rpm          = revolutions / seconds * 60

rpm = d * 60 * HAL_TICK_HZ / (2^32 * interval_ticks)
```

The unsigned subtraction followed by a signed reinterpretation gives the shortest-path difference across the wrap with no branch. A plain signed subtraction of two large positions either side of zero would report nearly a full revolution in the wrong direction.

interval_ticks is measured, never assumed. The clock-discipline loop trims the carrier to track the distributed clock, so the nominal 62.5 microseconds is not constant and differs slightly between joints in the same machine. A hard-coded period puts that trim straight into velocity as a per-axis scale error.

At 240 MHz and 15000 ticks per period the coefficient works out at $2.2352e-4$ rpm per position count. One encoder LSB is 8192 counts in this scale, so:

Averaging window	rpm per encoder LSB	Why
1 period (62.5 us)	1.83 rpm	Differentiation amplifies quantisation in inverse proportion to the interval.
4 periods (250 us)	0.46 rpm	Four consecutive differences telescope to the total displacement, so nothing is discarded - you get the noise floor of the slow rate and the fidelity of the fast one.

Which is why the ISR accumulates and the motion task divides. The ISR's own per-period estimate is used only for the over-speed threshold and the advance term, where a 1.83 rpm noise floor is irrelevant.

5. Angle advance

Two delays sit between measuring the angle and the resulting voltage appearing at the motor, and they add:

Delay	Size	Origin
Acquisition	20.31 us	The angle is latched at 10.94 us (TIM1 CCR4 = 2625) but the current is sampled at the carrier peak, 31.25 us.
Actuation	62.5 us	The duty computed now is preloaded and takes effect at the next reload, applying for the whole following period.

Both are proportional to speed in angle terms, so one per-rpm coefficient covers them:

```
advance = rpm * advance_per_rpm;      /* one integer multiply */
theta   = raw_angle + advance;         /* the add IS the wrap   */

advance_per_rpm = (POLE_PAIRS / 60) * 2^32 * T_delay
```

For 4 pole pairs and a total delay of 83 microseconds that is 23765. At 1499 rpm it gives 35.6 million counts, which is 2.99 degrees electrical - and the check is that the electrical frequency at 1499 rpm is 99.9 Hz, so 36000 deg/s times 83 us is 2.99 degrees.

The coefficient is tuned, not calculated. The figure above is a starting point. It also has to absorb the encoder's internal filter group delay and the modulator's own half-period offset, neither of which is known accurately. Section 8.1 gives the method: sweep the coefficient at constant speed and load, and take the value that minimises current. Minimum current at constant torque means maximum torque per amp, which means the angle is right.

6. Sine and cosine from one lookup

```
idx = theta >> 22;          /* top 10 bits: 1024 entries */
frac = theta & 0x3FFFFF;    /* remaining 22 bits      */

sin = tab[idx]      + (tab[idx+1] - tab[idx]) * frac >> 22
cos = tab[idx+256] + (tab[idx+257] - tab[idx+256]) * frac >> 22
```

Cosine is the same table read a quarter turn ahead - 256 entries of 1024 - so one 2 KB table serves both. The index masks wrap naturally, which is the accumulator representation paying off again.

Why interpolate rather than use a bigger table

A plain lookup quantises the angle to one table step. At 1024 entries that is 0.35 degrees electrical - but a 19-bit encoder on a 4 pole-pair motor resolves 0.0027 degrees electrical, so a plain lookup would discard almost all of the encoder's resolution and return it as torque ripple at the table-step rate.

Approach	Table size	Worst error	Cost per call
Plain lookup, 1024	2 KB	0.35 deg of angle	2 loads
Plain lookup, 32768	64 KB - half of DTCM	0.011 deg	2 loads
Interpolated, 1024	2 KB	1.49 LSB of Q15, 0.0045% FS	4 loads, 2 multiplies

Linear interpolation error is h^2 over 8, and at $h = 2\pi / 1024$ that is $4.7e-6$ - about 0.15 LSB in theory, 1.49 LSB measured across a thousand angles once table rounding is included. Two multiplies buy a factor of 32 in memory.

Returning **both** values from one call is not a convenience. The same pair must be used for the forward and the reverse transform, and handing back one struct makes that coherence structural. Two separate calls could silently be given different angles, and the symptom would be a torque error growing with speed - which reads exactly like a mistuned advance term.

7. Clarke, from two currents

The standard Clarke transform takes three currents. This one takes two, because phase A sits on the independent converter and cannot be sampled simultaneously with B and C.

```
For a three-wire motor:  i_a + i_b + i_c = 0

i_alpha = i_a              = -(i_b + i_c)
i_beta  = (i_a + 2*i_b) / sqrt(3) = (i_b - i_c) / sqrt(3)
```

Substituting the constraint removes phase A entirely. Nothing is lost: for a three-wire motor the third current is *determined*, so two measurements carry all the information there is. The result is zero skew where it matters, because B and C come from the guaranteed-simultaneous dual pair.

$1/\sqrt{3} = 0.57735$, which in Q15 is 18919.

Then why measure phase A at all? For the sum check. Its timing relationship to B and C is irrelevant when all you do is test whether the three add to zero, so full sensor redundancy costs nothing but a converter that was otherwise idle. A non-zero sum means a failed sensor, a saturated amplifier or a ground fault - a fourth path for current to leave by.

8. Park

```
i_d = i_alpha * cos + i_beta * sin  
i_q = -i_alpha * sin + i_beta * cos
```

A rotation by minus theta, taking the stationary alpha-beta frame into the rotor-aligned d-q frame. In steady state d and q are constants rather than sinusoids, which is what lets a simple PI regulate them to zero error.

Both operands are Q15 and the currents are milliamps, so each product is milliamps x 32768 and the shift back by 15 restores milliamps. Worst case is $82500 \times 32768 = 2.7e9$, which exceeds int32 - hence the 64-bit intermediate.

9. The current PI

```
limit = v_bus_mv / 2;                                /* per-axis headroom */

p_term = (error_ma * kp_q15) >> 15;                  /* mA -> mV      */
i_inc  = (error_ma * ki_q15) >> 15;

if (!clipped || integrator and i_inc have opposite signs)
    integrator += i_inc;

clamp integrator to +/- limit
out = p_term + integrator;
clamp out to +/- limit, set clipped
```

Error is in milliamps, output in millivolts, so the gains carry the units: k_p is millivolts per milliamp in Q15, which is ohms. That is not a coincidence - for an R-L load the analytically correct gains are $K_p = L \times \text{bandwidth}$ and $K_i = R \times \text{bandwidth}$, both of which have units of resistance.

The integrator is kept in output units rather than error units, so the anti-windup limit is the same number as the output limit. One clamp instead of two that have to be kept consistent.

The anti-windup clause that is easy to get wrong

Conditional integration freezes the integrator whenever the previous step saturated. On its own that is a trap: a loop that saturates once can never recover, because the integrator sits frozen at exactly the value keeping it saturated.

The second clause is the escape - integration is also permitted when the new increment would move the output back toward the linear region. Without it the drive latches into saturation and looks like a blown gate driver.

Why $v_{\text{bus}}/2$ and not $v_{\text{bus}}/\sqrt{3}$. With space-vector modulation the d-q vector magnitude can reach $v_{\text{bus}}/\sqrt{3} = 0.577 v_{\text{bus}}$ before the modulator saturates, so $v_{\text{bus}}/2$ is about 13% conservative. It is chosen anyway: it is one shift instead of a multiply, and the margin leaves room for the dead-time compensation that SVPWM adds afterwards.

10. Inverse Park and SVPWM

```
v_alpha = v_d * cos - v_q * sin    /* inverse Park      */
v_beta  = v_d * sin + v_q * cos

v_a = v_alpha                      /* inverse Clarke     */
v_b = -v_alpha/2 + v_beta * sqrt(3)/2
v_c = -v_alpha/2 - v_beta * sqrt(3)/2

offset = -(max(v_a, v_b, v_c) + min(v_a, v_b, v_c)) / 2

duty_x = ARR/2 + (v_x + offset) * ARR / v_bus_mv
```

$\sqrt{3}/2 = 0.86603$, which in Q15 is 28378.

Min-max injection is SVPWM

The textbook method computes a sector, then two vector dwell times, then maps them back to three duties. The four lines above produce **identical** duties, because that offset is precisely the common-mode term the sector method injects.

Why the common-mode term is free: only the *differences* between the three duties produce current in a floating star point. Adding the same amount to all three changes the applied vector not at all, while lowering the peak - which is what buys the 1.155x extension over sinusoidal modulation.

And it is branch-free, has no sector table, and cannot get a sector boundary wrong. That bug produces a torque ripple at six times electrical frequency, which is convincingly mechanical-looking and has cost people weeks.

The voltage-to-duty mapping

The bridge produces plus or minus $v_{\text{bus}}/2$ on a phase relative to the midpoint, so zero volts is 50% duty, not 0%. The centre term $ARR/2$ is that. A phase demand of $+v_{\text{bus}}/2$ maps to ARR and $-v_{\text{bus}}/2$ maps to 0, so the expression saturates exactly at the rails.

11. Dead-time compensation

During the dead time both devices in a leg are off, and the phase voltage is decided by the **direction of the current**, not by the commanded duty - the body diode that conducts is the one the current forces on. So the applied volt-seconds fall short of the commanded value by one dead time, with a sign set by the current direction.

```
if      (i_x > DEADBAND) duty_x += DEAD_TIME_DTG;
else if (i_x < -DEADBAND) duty_x -= DEAD_TIME_DTG;
```

A fixed number of counts, not scaled by duty or speed - the error is one dead time regardless of operating point.

The deadband is the important part. Near zero current the sign is noise, and a compensation that flips with it injects a square wave at whatever rate the noise happens to have. That is far worse than no compensation, and it is the usual reason dead-time compensation makes a drive rougher rather than smoother.

12. The duty clamp

Low-side sensing forces a ceiling on duty: the shunts only carry phase current while the low-side devices conduct, and the sampling aperture has to fit inside that window with the amplifier settled.

```
CCR_MAX = ARR - DEAD_TIME_DTG - INA241_SETTLE - APERTURE/2
         = 7500 - 60 - 240 - 33 = 7167          (95.6% duty)
```

When the maximum exceeds it, `hal_pwm_set_duty` applies a **common-mode shift** first:

```
excess = max - CCR_MAX;
if (min >= excess)  a -= excess; b -= excess; c -= excess;
else               scale all three by CCR_MAX/max;
```

Subtracting the same amount from all three lowers the peak **without altering the applied vector at all**, for the same reason the min-max injection works. It is free and exact. Scaling loses magnitude and is the fallback for when the shift would drive a phase below zero - which means the modulator asked for something the sensing window cannot express, and the fix belongs upstream.

Clamping each phase independently would be the obvious implementation and is wrong: it distorts the vector and injects a harmonic that grows with modulation depth.

13. Worked example: one period, entry to exit

A joint running steadily at 1500 rpm, 4 pole pairs, 48 V bus, commanded 5000 mA of q-axis current. Every number below is computed with the same integer arithmetic the firmware uses, including the truncations.

Entry

Step	Computation	Result
hal_deadline_kick()	TIM6 CNT = 0, CEN = 1	monitor rearmed
t_entry = hal_time_now()	TIM5->CNT	0x1A2B3C4D
delta = t_entry - last	wrap-safe unsigned subtract	15002 ticks = 62.508 us
interval check	15002 vs 15000 +/- 3750	within tolerance
hal_carrier_position ()	TIM1->CNT, past peak, counting down	7320, expected 7300 +/- 600

Acquisition and scaling

Channel	Raw	minus offset	x 165000 >> 16	Sign inverted
Phase A (ADC3)	30784	-1986	-5001	+5001 mA
Phase B (ADC1)	33763	+993	+2500	-2500 mA
Phase C (ADC2)	33763	+993	+2500	-2500 mA
Bus (ADC1 rank 2)	47663	-	-	48000 mV

The three currents sum to **+1 mA**, not zero. That is the 2.52 mA quantisation of a single ADC count showing up, and it is exactly why current_sum_tol_ma exists rather than a test for equality. A tolerance set from measurement will sit at some tens of milliamps; a tolerance of zero would fault every period.

Angle

Step	Computation	Result
Encoder process data	19-bit right-aligned	0x60000 (example)
Left-align	pd << 13	0xC0000000
Electrical	position x 4 + offset	0x00000000 x 4 wraps - use 0xC0000000
Speed	d = 6709248 counts over 15002 ticks	1499 rpm
Advance	1499 x 23765	35623735 = 2.99 deg electrical
theta (after advance)		0xC0000000 = 270.0 deg
Table index	theta >> 22	768, frac = 0
sin, cos	tab[768], tab[1024 wraps to 0]	sin = -32767, cos = 0

270 degrees is chosen for the example because it makes the transform arithmetic checkable by eye: the current vector lies along phase A, and at theta = 270 that is entirely q-axis.

Transforms

Step	Computation	Result
Clarke alpha	$-(i_b + i_c) = -(-2500 + -2500)$	5000 mA
Clarke beta	$(i_b - i_c) \times 18919 \gg 15 = 0 \times 0.577$	0 mA
Park d	$(5000 \times 0 + 0 \times -32767) \gg 15$	0 mA
Park q	$(-5000 \times -32767 + 0 \times 0) \gg 15$	4999 mA

i_q comes back 4999 rather than 5000: the Q15 sine is 32767, not 32768, so the round trip loses one part in 32768. Below the 2.52 mA measurement resolution, and it does not accumulate.

Control

Step	Computation	Result
Error q	$5000 - 4999$	1 mA
P term q	$1 \times 16384 \gg 15$	0 mV (below one millivolt)
Integrator q	settled: back-EMF plus $I \times R$	21000 mV
v_q	$0 + 21000$, under the 24000 mV limit	21000 mV
v_d	cross-coupling compensation	-1200 mV

At steady state the proportional term is essentially zero and the integrator carries the whole output - which is what it is for. The 21 V is the back-EMF at 1500 rpm plus the resistive drop at 5 A.

Modulation

Step	Computation	Result
Inverse Park alpha	$(-1200 \times 0 - 21000 \times -32767) \gg 15$	20999 mV
Inverse Park beta	$(-1200 \times -32767 + 21000 \times 0) \gg 15$	1199 mV
v_a	$= v_{\alpha}$	20999 mV
v_b	$-10499 + (1199 \times 28378 \gg 15) = -10499 + 1038$	-9462 mV
v_c	$-10499 - 1038$	-11538 mV
offset	$-(20999 + -11538) / 2$	-4731 mV
duty a	$3750 + (20999 - 4731) \times 7500 / 48000$	6291
duty b	$3750 + (-9462 - 4731) \times 7500 / 48000$	1532
duty c	$3750 + (-11538 - 4731) \times 7500 / 48000$	1207

Output

Step	Computation	Result
Dead-time comp a	$i_a = +5001 > 200$, so +60	6351
Dead-time comp b	$i_b = -2500 < -200$, so -60	1472
Dead-time comp c	$i_c = -2500 < -200$, so -60	1147
Duty clamp	max 6351 vs CCR_MAX 7167	not active
Written	CCR1, CCR2, CCR3	6351, 1472, 1147

Sanity checks on the result

- Peak phase voltage 20999 mV against the modulation ceiling $v_{\text{bus}}/\sqrt{3} = 27713$ mV gives **76% modulation** - working hard but with headroom.
- Duty a is above 50% and b and c below, which is right: phase A carries positive current, so its high side conducts longer.
- Duty b and duty c are nearly equal and i_{beta} is nearly zero, consistent with a current vector aligned to phase A.
- Maximum duty 6351 of 7500 is 84.7%, comfortably under the 95.6% clamp, so the low-side sampling window is intact.

Housekeeping before exit

Step	Computation	Result
Velocity accumulate	<code>d_position += 6709248, d_ticks += 15002</code>	period 3 of 4
Publish feedback	<code>angle, positions, i_d, i_q, v_bus, stamp</code>	double buffer swapped
Pend motion	<code>every 4th period</code>	NVIC pending bit set
Re-arm encoders	<code>clear EOT, clear TC, reload NDTR, cycle CS</code>	chain armed
Scope log	<code>8 selected channels written</code>	<code>isr_ticks</code> measured across all of it

On the fourth period the accumulator publishes 26836992 counts over 60008 ticks, and the motion task's single divide returns **1499 rpm**.

14. Overflow analysis

Every multiply in the ISR, with its worst-case operands. int32 holds 2.147e9; anything above that needs a 64-bit intermediate, and the compiler will not warn at run time.

Expression	Worst case operands	Product	Fits int32?	Written as
delta * MA_PER_COUNT_Q16	32768 x 165000	5.4e9	No	int64 intermediate
raw * VREF * DIV	47663 x 3300 x 20	3.1e9	No	int64 intermediate
i_alpha * cos	82500 x 32768	2.7e9	No	int64 intermediate
error_ma * kp_q15	82500 x 32768	2.7e9	No	int64 intermediate
v_beta * SQRT3_2	27713 x 28378	7.9e8	Yes	int32 fine
(v + off) * ARR	40000 x 7500	3.0e8	Yes	int32 fine
d * 60 * TICK_HZ	2 ³¹ x 1.44e10	3.1e19	No	int64, and even that is tight
rpm * advance_per_rpm	24000 x 23765	5.7e8	Yes	int32 fine
(b - a) * frac	65534 x 4194303	2.7e11	No	int64 intermediate

The speed calculation is the tight one. d can be plus or minus 2³¹ and the numerator multiplies it by 1.44e10, giving 3.1e19 against an int64 ceiling of 9.2e18 - so a full-scale d WOULD overflow. It never occurs: d is bounded by the maximum position change in one period, which at the 12000 rpm encoder limit is about 5.4e7, giving 7.7e17 with an order of magnitude to spare. Worth knowing that the margin comes from physics rather than from the types.

On the M7, a 64-bit accumulate is the SMULL or SMLAL instruction and takes one cycle, so widening where it is needed costs essentially nothing. The reason not to widen everything is readability, not speed.

15. Precision budget

Where accuracy is actually lost, largest first.

Source	Magnitude	Comment
Current measurement quantisation	2.52 mA per count	Sets the floor. Everything below this is noise in the arithmetic rather than in the answer.
Encoder absolute accuracy	about 20 arcsec	2 LSB referenced to 12 bits of one sine period. Eight times coarser than the 2.5 arcsec output resolution - build the cross-check threshold from this, not from resolution.
Trig table interpolation	1.49 LSB of Q15, 0.0045%	Measured across a thousand angles.
Q15 round trip through Park	1 part in 32768	Visible as i_q returning 4999 for 5000. Does not accumulate.
Bus voltage divide-before-multiply	20 mV	A defect, not a limit - see section 2.
Velocity quantisation, 4 periods	0.46 rpm	One encoder LSB over 250 us.
Angle advance model error	grows with acceleration	The term is exact only at constant velocity; under acceleration the compensation lags. Nothing corrects this, and nothing needs to at the accelerations a geared joint reaches.

The ordering matters when deciding where to spend effort. Widening the trig table would improve a term that is already 500 times below the current measurement floor. Fixing the bus voltage expression takes one line and removes an error twenty times the converter's own.

16. Checks you can run on the bench

Each of these catches a specific class of error, and all of them work with the scope buffer and no other equipment.

Check	Method	What it catches
Current sign	Force a known duty at a forced angle. Phase with the highest duty must read positive current.	An inverted CURRENT_SIGN - the single most common low-side-sensing error, and it makes the drive run away rather than fail.
Current scale	Series ammeter against the logged i_a at a fixed forced angle.	A wrong shunt value or INA241 gain. A factor of two here looks like a tuning problem for weeks.
Sum check residual	Log i_a + i_b + i_c at various operating points; it should stay within a few counts.	A drifting offset, a saturating amplifier, a ground fault.
Transform round trip	Log i_d and i_q at constant torque and constant speed. Both must be nearly DC.	A wrong pole-pair count, a wrong commutation offset, or a sine table index error - all of which make d and q sinusoidal instead of flat.
Angle advance	Sweep advance_per_rpm at constant speed and load, log i_q.	The minimum is the correct value. A monotonic curve with no minimum means the sign is inverted.
Modulation ceiling	Log duty_max while ramping speed.	If it reaches CCR_MAX before rated speed the bus is too low or the clamp is too tight.
Velocity noise floor	Stand still, log the motion task's velocity output.	It should sit within about +/- 0.5 rpm. Larger means the accumulator is being cleared wrongly or dt is coming from a constant.
ISR duration spread	Histogram isr_ticks over a long run.	Near-zero spread. Visible spread means something still executes from flash, or a HAL dispatcher is in the path.

17. The constants this all rests on

Nine numbers determine every result above. Seven are still placeholders.

Constant	Used by	Status
VREF_MV	current and bus scaling	3300, nominal - measure it
SHUNT_MILLIOHM	current scaling	placeholder
INA241_GAIN	current scaling	placeholder
VBUS_DIV_NUM / DEN	bus scaling	placeholder
MOTOR_POLE_PAIRS	electrical angle, advance	placeholder
ENC_RESOLUTION_BITS	left-align shift, velocity scale	from MPC - unknown
COMMUTATION_OFFSET	electrical angle	found at stage 5
DEAD_TIME_DTG	dead-time comp, CCR_MAX	60, confirm against the driver
INA241_SETTLE_TICKS	CCR_MAX	placeholder - measure at stage 3

Every placeholder above carries a #warning in config/, so the build tells you which are still guesses on every compile. They are not defects to be silenced - they are the measurement list, and the worked example in section 13 is arithmetically correct but numerically provisional until they are closed.